

Project Report for Final Project
Modern Application Development – I

Student Name : Atul Mishra

Roll No. : 23f2001262

23f2001262@ds.study.iitm.ac.in

I am a student in the IIT madras . I enjoy learning about programming and new technologies. This project is also a lot of learning and fun for me. In this project, I learned a lot about the technologies that I used to build it.

Description :

In this project , I have to use HTML, CSS, Bootstrap for frontend , flask and flask -restful used for routing and programming(functionalities) and flask – sqlalchemy for data storage , Matplotlib for graph generation and other necessary module to build Household service app.

I build models by using flask-sqlalchemy that made database with SQLite for storing data about admin, professionals, customers and service request and review(rating or review) ,services.

I build the index page where a short description are there and login and signup pages and dashboard for all users type with the necessary pages like search , review summary etc.

In this app admin can create/edit/modify services based on that professional and customer can provide and book services respectively . Admin can block/unblock/delete other user and also search service , service request and other users. Admin can also view professional details like verification docs, and their other details like review or rating etc. based on that he or she can accept/ block /reject them. Admin can see the details of customers also. Admin summary include the overall rating , service request graph and user graph .

Professionals can provide one service only . They can accept and reject multiple service requests (if they accept a service request all the service request that overlap got rejected) that they received from the customers and view the detail like address phone of customer and their service request details date of completion and requested any remarks. They can search the requests from all the request they received and see summary how well they are performing by their rating and service request graph.

Customers can book many services and close the services, edit the services. If he close the service he has to fill the review form. They can also search the services by the name, location and pincode. They can see summary (service requested , accepted and rejected) .

Technologies used :

Flask : used for building web application

Flask-SQLAlchemy : extension of flask, used to handle database connection across the app

Datetime : the python module is used for time stamping the class operation

Flask-Restful : used for creating api .

Matplotlib : used for creating graph.

os : used for interaction with os provide feature like create folder deleting and many more.

DB Schema Design:

Admin : have 3 attribute with id as their primary key

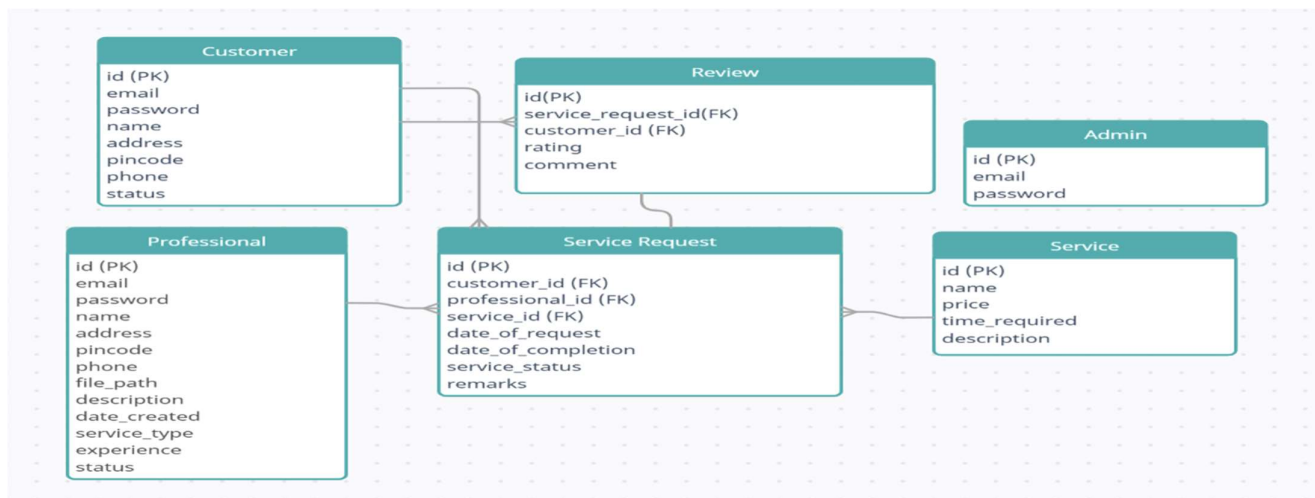
Customer: have 8 attribute with id as primary key , a customer can have many service request but a service request have only one customer so one-to-many relationship between customer and service request and one-to-many relation with review.

Professional: have 13 attribute with id as primary key and one to many relationship same as customer.

Service Request: have 8 attribute with id as primary key and 3 foreign key (customer_id , professional_id and service_id) and one-to-one relation with review (one service request have one review.)

Service: have 5 attribute with id as primary key and one-to-many relationship with service request (one service have many service request but one service request does not have many service)

Review : have 5 attribute with id as primary key and 2 foreign key (customer_id and service_request_id).



API Design :

- **AdminApi** : implemented using get, post, delete and put method. Get fetch all the data of the admin database. Post for adding new data in the admin database , put for update particular data and delete for delete the particular data from the admin database.
Endpoint : `"/api/admins"` , `"/api/admins/<int:id>"`
- **ProfessionalApi**: implemented using get, post, delete and put methods . same as admin
Endpoint : `"/api/professionals"` , `"/api/professionals/<int:id>"`
- **SearchProfessionalApi**: implemented using get method it takes id to search particular professional.
Endpoint : `"/api/search_professional/<int:id>"`
- **CustomerApi**: implemented using get , post, put and delete methods. Same as admin.
Endpoint: `"/api/customer"` , `"/api/customer/<int:id>"`

Architecture and features :

Project is code is organized based on its utility in different files. I named my project House-service

Inside this folder, there are a python file name app.py which is used for setup and running the app, requirements.txt for all the requirements for this app and 5 folders named backend (contains 3 python files : **useful_function** for supportive function , **controller** for views routing and essential functions and **modals** for all the model and relationship.) , instance (contain the database file.) , static (for CSS and images(contains 3 folder as admin, professional, customer for storing their summary graph)) , templates (for all the html files) , professional_verification (for the documents uploaded by the professional for verification).

Controllers that were used:

- `@app.route("/")`
- `@app.route("/login", defaults={"msg": ""}, methods=["GET", "POST"])`
`@app.route("/login/<msg>", methods=["GET", "POST"])`
- `@app.route("/<registerer>/register", methods=["GET", "POST"])`
- `@app.route("/admin_dashboard")`
- `@app.route("/customer_dashboard/<user>")`
- `@app.route("/professional_dashboard/<id>")`
- `@app.route("/dashboard/admin/search", methods=["GET", "POST"])`
- `@app.route("/customer_dashboard/<user>/search", methods=["GET", "POST"])`
- `@app.route("/professional_dashboard/<user>/search", methods=["GET", "POST"])`
- `@app.route("/logout")`
- `@app.route("/admin/add_service", methods=["GET", "POST"])`
- `@app.route("/admin/<task>/service/<id>", methods=["GET", "POST"])`
- `@app.route("/admin/show_detail/<item>/<id>")`
- `@app.route("/professional_view/<filename>")`

- `@app.route("/customer_dashboard/<user>/particular_service/<service_type>")`
- `@app.route("/customer_dashboard/<user>/book_service", methods=["GET", "POST"])`
- `@app.route("/view/<user>/profile/<id>", methods=["GET", "POST"])`
- `@app.route("/admin/<user>/<id>/<status>")`
- `@app.route('/professional_dashboard/<user>/service/<request_id>/<status>')`
- `@app.route("/summary/<user>/<id>")`
- `@app.route('/customer_dashboard/edit_request/<user_id>/<request_id>', methods=['GET', 'POST'])`
- `@app.route('/service_request_detail/<user>/<uid>/<rid>')`
- `@app.route('/customer_dashboard/<uid>/<rid>', methods=['GET', 'POST'] )`
- `@app.route('/customer_dashboard/<uname>/delete/<rid>')`

Video :

Link: https://drive.google.com/file/d/1vU9rujltM52dFCn46763Cusv7_AWeHtY/view?usp=sharing

1. 10 second intro and about video.
2. 8 minute 46 second demonstration and explanation .